detection. Once we map it to a standard NLP task, it allows us to leverage the state-of-the-art models already built for that task in NLP. Similarly, we can model the problem of automatic comment generation as a machine translation task.

2. For each of these software engineering tasks, we need to consider how we can generate training data. For example, consider the problem of automatic bug detection. Let us assume that we have modelled this as a two-class text classification problem, where given a piece of code snippet, we label it as either positive sample (correct code) or negative sample (buggy code). While there are a large amount of code samples that are publicly available, the assumption is that most of the publicly available code is correct. This enables us to get a large number of positive samples. But how do we get negative samples – examples for buggy code? One way is to have a human annotator go through the code samples and find each and every piece of buggy code. However, this will require a lot of effort and will not be scalable, and it is highly possible that there may not be sufficient examples of buggy code in open source repositories. An alternative approach is to automatically create buggy code from correct code using simple code transformations. For example, consider detecting simple bugs where arguments to a function are mistakenly swapped. Here is a small example of a swapped arguments bug:

```
Void setDim(int x, int y) {}

Int Xdim = …
Int Ydim = ..
setDim(y,x);
```

   In the above example, we have the first and second argument mistakenly swapped at the point of invocation. If we want to create training data for classifying this kind of bug, we can take correct function calls and do a simple program transformation, which swaps the function call arguments to create buggy examples. Hence, it is possible to synthetically generate data for creating large amounts of training data.
3. The final question we need to answer when applying DL/NLP techniques to code is: How do we represent the code? There are well known methods of representing text such as using word embeddings (word2vec, glove embeddings), sentence embeddings and contextual embeddings (ELMO, BERT embeddings), etc. Can we represent code also using similar techniques? Should code be treated as a textual stream of tokens? How do we represent dependency information between variables, control flow graph, etc, as part of code representation? Unlike human text, where the information is solely contained in the text, programs/code can have additional information that can be inferred based on their control flow and data flow. Hence this auxiliary information also needs to be represented in modelling source code. There has been considerable work in modelling source code using deep learning techniques. There is a recent survey paper that covers these techniques in detail available at *https://arxiv.org/pdf/2002.05442.pdf*.

Another thing we should keep in mind when we consider applying DL techniques to source code analysis tasks is the following: What is the input domain and the output domain of the task? For example, consider the problem of deep code search. The input is a user query in natural language text. Output is a piece of code snippet in a high level programming language. The input domain text captures the high level intent of the programmer in natural language. The output domain text captures the low level implementation details in code. Often, the surface level similarity (both lexical and syntactic) between input and output text in this task would be negligible. Hence any efficient method for deep code search needs to keep this representational mismatch in mind. One way of overcoming this issue is to project both the user query and code snippet into a common vector space representation. This idea is well described in the paper 'Deep Code Search' available at *https://guxd.github.io/papers/deepcs.pdf*.

One major challenge in applying probabilistic deep learning techniques to code is: How can we ensure correctness? For example, consider the problem of program repair or automatic bug fix. Given a faulty piece of code, we can build a deep learning model which generates the correct code. This problem can be modelled similar to the neural machine translation or grammatical error correction problem in NLP. However, in case of source code, the fixed code needs to be exactly correct in all tokens so that it can be compiled without errors by the compiler and get executed. Ensuring and verifying correctness is a major challenge in such situations. One way of working around this problem is to have an external oracle, which can verify the correctness of the generated code. This idea is explored well in the paper 'Deep Fix' available at *http://www.iisc-seal.net/deepfix*. They use an external oracle, namely the compiler itself, to check whether the generated code is correct or not. However, the compiler can check only the lexical and syntactical correctness. The logical or semantic correctness cannot be ensured by it. Hence this still remains an open challenge in applying probabilistic deep learning techniques to code generation. In next month's column, we will get into some of the applications of deep learning techniques in compiler optimisations.

Feel free to reach out to me over LinkedIn/email if you need any help in your coding interview preparation. If you have any favourite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at *sandyasm_AT_yahoo_DOT_com*. Wishing all our readers happy coding until next month! Stay healthy and stay safe. END

By: Sandya Mannarswamy

The author is an expert in natural language processing and is currently working as an independent researcher. Her interests include natural language processing, machine learning and AI.